SNC Architecture Contracts and Validation Handoff

Engineering addendum for the SNC development agent

3 November 2026 | Version 1.0 | Prepared for Cody

Purpose and requested outcome

Review the contracts in this addendum against the current SNC implementation, identify which guarantees already have evidence, and propose the smallest useful experiments for the remaining gaps. The immediate goal is an implementation map and a prioritized validation plan that preserve SNC's established architecture.

SNC promises durable identity and state while machines, owners, software versions, and execution resources change underneath them. That promise requires explicit behavior at the boundaries between persistence, scheduling, security, external effects, and recovery. This document turns the seven recommendations discussed with Cody, plus seven additional contracts, into a concrete engineering review.

Architecture to preserve

SNC remains a general-purpose, Linux-hosted, state-native server platform. Stable ObjectIDs identify state independently of its physical location. Pinned workers own shards; authoritative mutation happens at the owner. Documents, events, timers, activation, and security are generic platform capabilities. The Agent Runtime is one framework above those capabilities. SNC Server OS and TM87 remain independently deployable.

Architecture and Research Vision v0.2 reports a C++ foundation with WAL and recovery, replication, fencing, failover, placement epochs, mint leases, migration, and crash injection. Rust is the intended production direction, with C++ retained as a behavioral reference. Treat that as the reported baseline and attach current repository evidence during review. The v0.2 document is reference [1].

Recommended first decisions

- Define what a successful acknowledgment guarantees and which operation histories are legal under failure.
- Select compatibility behavior for an entity that resumes under a different software version.
- Specify how an external action with an uncertain outcome is reconciled before another attempt.

All contracts below are proposals for review. They do not establish that a feature is absent or override existing project decisions. Retain current work slices, manifests, and independent implementation, review, and QA roles. PostgreSQL is a relevant comparison and integration topic; this addendum does not assign it authority over SNC's object substrate.

Contract priorities and dependencies

The first seven IDs correspond to the earlier discussion. R08 through R14 are additional recommendations. “Define now” means settle the boundary before dependent APIs become fixed; it does not require a complete implementation immediately.

ID	Contract and main question	Sequence
R01	Software evolution Which code can resume which durable state?	Define now
R02	External effects How are unknown outcomes and retries reconciled?	Define now
R03	Execution scheduling What can run while an entity awaits a result?	Define now
R04	Read consistency What does each query or snapshot guarantee?	Define now
R05	Security boundaries Where are identity and authority checked?	Define now
R06	Disaster recovery How is a coherent cluster restored safely?	Validate core
R07	Reproducible verification Can a failing history be replayed and checked?	Extend harness
R08	Commit acknowledgment Which failures may follow a successful response?	Define first
R09	Cancellation and time How are late work and overdue timers handled?	With activation
R10	Bounded work How does the system behave under sustained overload?	With scheduler
R11	Retention and deletion When can history and deduplication evidence be removed?	Before collection
R12	Protocol compatibility Which mixed versions and stored formats interoperate?	With Rust port
R13	Module containment What prevents extensions from harming the core?	Before plugins
R14	Durable activation How does a committed change become runnable work?	With events

Define R08 first. Settle R01 and R12 before software migration, and R05 and R13 before extensions. Validate R02, R09, and R11 together for retry safety; connect R04 and R14 for consistent reads and activation.

Commit semantics and external effects

R08 Define the acknowledgment boundary

A successful response should identify a documented durability and consistency policy. The following are distinctions to resolve, not proposed product mode names or claims about supported behavior. The receiving agent should map them to SNC's actual API and WAL protocol.

Boundary	Question the contract must answer
Accepted into a queue	Has work merely been admitted, or has state changed? What survives a process crash?
Committed locally	Which state, result, deduplication record, and pending effects commit together?
Persisted to local storage	What flush and device assumptions establish survival after host power loss?
Acknowledged by replicas	Are replicas acknowledging receipt, durable storage, or application? Which membership and failure model apply?

Specify the ordering or linearization point for each supported operation, the result of retrying after a lost response, and the response when authority or required durability is unavailable. Physical colocation alone does not make a group of updates transactional. Name the actual atomicity domain. PostgreSQL's reliability documentation illustrates why operating system and device caches matter to the persistence claim [6].

R02 Record external intent and uncertain outcomes

An SNC-native TM87 operation can succeed on the appliance and then lose its acknowledgment before SNC records completion. An effect record should retain a stable operation ID, target, request fingerprint, applicable execution contract, retry policy, and outcome evidence. Commit the intent with the relevant local transition, then dispatch through an executor. Record receipts or readback evidence when available.

Use states such as pending, in flight, succeeded, failed, and outcome unknown. A dispatch lease coordinates executors but cannot revoke a packet already sent. Destination-supported idempotency keys, conditional writes, or fencing can prevent duplicate or stale effects where the destination implements those semantics. Otherwise, reconcile using authoritative readback or explicitly surface the ambiguity. Compensation is a new action with its own failure modes.

Internal atomic commit and destination deduplication address different boundaries. DBOS explains this distinction in its engineering discussion of transactional execution [3]. An unknown result must not be converted to success, failure, or a blind retry merely to finish recovery.

Experiment E02: crash after the external system commits but before SNC stores its receipt. Reuse the same operation identity during recovery. Test both a destination with deduplication and one without it; verify the declared reconciliation behavior in each case.

Software evolution and protocol compatibility

R01 Let durable entities outlive code versions

Versioned document schemas and Execution Contracts are a useful foundation, but the compatibility boundary must also cover operation implementations, pending events, timer payloads, continuations, indexes, and stored results. Give each durable operation enough identity to resolve the correct behavior after recovery. Specify which changes can resume old work directly, which require a migration, and which require an older compatible executor.

A suspended entity should resume at a declared durable boundary. Decide whether that boundary is an explicit state machine, replay of recorded steps, or another representation. Persisting application state does not automatically preserve a native stack, an open socket, or an arbitrary closure. If replay is used, record or otherwise control nondeterministic inputs such as external results, random values, and time observations.

Test mixed versions during rolling upgrades. Describe the point after which rollback is no longer supported and the recovery procedure for an interrupted schema migration. Do not silently reinterpret old state with a new layout. DBOS's official workflow upgrade documentation provides a reference for versioning and patching, rather than proof that either mechanism is sufficient for SNC [2].

R12 Separate durable formats from runtime layouts

Define versioned storage, replication, migration, and client protocol envelopes. An envelope should identify the format and operation, describe required capabilities, and make unsupported mandatory behavior explicit. Define how unknown optional fields are preserved or discarded, how bounds are validated, and which changes require a feature gate across the cluster.

C++ and Rust need a shared semantic contract; they do not need identical internal representations. Choose durable layouts deliberately rather than relying on compiler ABI, raw pointers, native padding, or unspecified byte order. A directly mapped durable format is possible, but its layout and upgrade rules become a maintained interface. ObjectID indirection should remain valid across address-space and placement changes.

A parser accepting a newer message does not prove that it understands the message's semantics. Maintain an explicit compatibility matrix for readers, writers, protocol peers, and recovery tools. Protocol Buffers documents useful distinctions between safe, unsafe, and conditionally compatible changes [7]; this is a design reference, not a recommendation to adopt that encoding.

Experiment E01: create state and suspended work under v1, run a mixed v1/v2 cluster, migrate ownership, and resume. Compare C++ and Rust behavior where both implementations exist. Test the rejected combinations as carefully as the accepted ones.

Execution cancellation and timer semantics

R03 Separate entity waiting from worker occupancy

Pinned ownership should govern authoritative state mutation. Slow model inference, remote I/O, blocking plugin calls, and heavy computations should execute outside the short mutation turn. Persist the request or continuation boundary, release the owning worker, and return a completion message through the authoritative owner. Define memory ownership for any data passed to an external executor.

Decide separately whether other operations may run on the same entity while it is waiting. An entity can retain exclusive logical execution while its worker serves other entities. If interleaving is allowed, validate preconditions when the continuation resumes, because state may have changed. Bound execution turns and avoid holding an owner-local transaction across remote I/O. Orleans documents the corresponding scheduling and reentrancy tradeoffs [4].

An operation token should identify the logical operation and its current generation. Owner authority must be checked through SNC's existing placement and HA rules. Do not mechanically discard all results from an earlier owner: recovery may legitimately adopt an already completed external effect. Distinguish a stale attempt from an effect that remains part of the current operation.

R09 Make cancellation and time durable semantics

Cancellation is a request to change the operation's permitted future behavior. Define when it becomes durable, which outstanding children it covers, and what happens when cancellation races with completion. A late result can remain audit evidence while being ineligible to mutate current state. External actions already performed require reconciliation; changing a local status cannot undo them.

Concern	Contract to specify
Timeout	Whether it stops waiting, revokes permission to commit, requests cancellation, or combines these actions.
Elapsed deadline	Which local monotonic clock is used while running, and how remaining work is reconstructed after restart.
Scheduled wall time	How timezone rules, clock corrections, missed deadlines, and failover affect eligibility.
Recurring work	Whether downtime causes catch-up, coalescing, skipping, or a bounded number of overdue activations.
ANY or ALL waits	How a winner is committed and how late signals, cancellation, and re-registration are handled.

Experiment E03: delay a completion, commit cancellation or change the plan, then fail over and deliver the result twice. Verify the intended logical outcome and preserve useful evidence. Add clock jumps and a backlog of overdue timers once timer semantics are implemented.

Read consistency and durable activation

R04 Give each read path a clear guarantee

Define the atomicity domain for mutations and the consistency model for reads separately. Search candidates, authoritative object reads, multi-object snapshots, and administrative inspection may legitimately provide different guarantees. A candidate returned by an asynchronous index must not automatically become authority for a permission decision or state-changing precondition.

Read path	Required explicit distinction
Owner read	Current leader read, replica read, or version-specific read; routing and stale-epoch behavior.
Index or search	Lag, false omissions, deleted candidates, and whether owner readback is required.
Corpus manifest	The exact object and schema versions supplied to a model, with their provenance.
Consistent snapshot	A defined consistent cut across participating owners, if the API promises one.
Recovered snapshot	The retained versions, indexes, policy material, and keys needed to reproduce the read.

Reproducibility and cross-object consistency are separate properties. Pinning exact immutable versions makes a corpus reconstructable; it does not by itself prove that the selected versions formed a mutually consistent state. Define which applications need that stronger property and how the system establishes it. A wall-clock timestamp alone is insufficient as a cross-owner snapshot protocol.

R14 Commit state changes and activation obligations coherently

A process can crash after committing an object change but before publishing its event. A wait registration can race with the change that should wake it. Specify an atomic or recoverable connection between the authoritative transition and its activation obligation. Candidate designs include a durable outbox in the same commit domain or consumption of committed WAL records; choose based on the existing implementation.

For conditional waits, define a registration and recheck protocol tied to a version or sequence boundary so a qualifying change cannot fall into the gap. Delivery may be at least once, but duplicate activation must obey the operation’s effect rules. Durable obligations need retention until consumption or another terminal disposition is established.

Treat secondary indexes as either authoritative structures with transactional obligations or derived projections with a declared lag and rebuild path. Expose a progress watermark where a consumer needs to know what has been indexed. Rebuilds and catch-up work must respect admission budgets.

Experiment E04: interrupt every boundary between mutation, event persistence, dispatch, consumption, and acknowledgment. Race condition registration against the triggering mutation. Verify that committed obligations remain recoverable and that duplicate delivery does not violate the declared state-transition invariant.

Security and module containment

R05 Establish identity and authorization in core interfaces

The source document describes security as foundational in Section 17 but places the Security Fabric after documents and activation in Section 24. Resolve that sequencing mismatch by defining principal identity, security-domain ownership, capability propagation, and owner-side authorization before those interfaces become stable. The full administration layer can be implemented incrementally.

Carry authority through queued work, migrations, retries, callbacks, and restore. Define when policy is evaluated: admission, dispatch, state commit, or an explicit combination. A pinned historical policy can explain an execution without necessarily granting permission under a later revocation. Make that distinction visible in the execution and audit model.

Authorization must cover the exact target, operation, scope, and version-sensitive preconditions where relevant. Record denied actions as well as permitted ones at an appropriate audit level. Secret references should resolve through the broker at the permitted boundary. Restore and replay need explicit rules for expired, revoked, or missing credentials.

R13 Classify extensions by their isolation requirements

An ownership API can enforce correct access by cooperating code; it is not, by itself, a security boundary against arbitrary code in the same process. Rust memory safety helps with one class of faults, but plugin code can still block, exhaust resources, issue unintended I/O, or cross an unsafe FFI boundary. Select isolation according to the module's trust and capabilities.

Module category	Proposed execution boundary
Trusted core component	May share the runtime where its latency, memory, and authority contracts are audited.
Blocking or failure-prone adapter	Use a bounded executor or process boundary; return messages rather than direct state mutation.
Untrusted extension	Use an enforceable sandbox or process isolation with narrow brokered interfaces and explicit limits.

Do not add an entire extension platform solely to satisfy this table. First classify the modules that actually exist or are already planned, then make their authority and failure containment explicit. Background tasks must retain the initiating security domain instead of inheriting unrestricted core authority.

Experiment E05: revoke a capability while a request waits, move its entity, then deliver the old request and callback. Include a cross-tenant target and a stalled adapter. Verify the selected revocation policy, tenant separation, rejection of unauthorized mutation, and continued progress for unrelated entities.

Recovery retention and deletion

R06 Restore a coherent system after a bad history

Replication and failover protect against specified machine or replica failures. Disaster recovery also addresses a bad migration, accidental deletion, or corruption that reaches every replica. Define recovery point and recovery time objectives for the supported durability policy, and verify them through restore drills rather than inferring them from replication tests.

A restore manifest should identify object snapshots, WAL ranges, schemas, placement and authority metadata, pending operations, activation state, and required security material. Establish a coherent recovery boundary for authoritative data; rebuildable indexes can follow their documented catch-up rules. Restore into isolation first, with external dispatch disabled.

Ensure the restored cluster cannot compete with an old live cluster for authority. Reconcile control-plane membership, leases, and fencing through the supported recovery protocol. Historical ObjectIDs must remain meaningful, while future allocation must avoid reusing identities that escaped after the restored checkpoint. Define how allocation high-water marks or an equivalent incarnation mechanism preserve that property.

External systems will not necessarily roll back with SNC. Before enabling dispatch, reconcile pending and previously completed effects against the restored history. An isolated restore should make uncertainty visible rather than reissuing old commands automatically.

R11 Define retention for state and correctness evidence

Long-lived identity does not require retaining every byte forever. Specify roots that keep objects or versions reachable: active entities, pending effects, timers, snapshots, audit requirements, and explicit holds. Define tombstones and stale-message handling so delayed retries cannot recreate a deleted entity accidentally.

Deduplication records and tombstones are correctness state. Their retention must cover the supported retry and replay horizons, including delayed replicas and disaster recovery. If that horizon is unbounded, bound acceptance through operation generations or another explicit rule; do not assume a short cleanup interval is safe.

The v0.2 preference for additive knowledge consolidation should become a default with declared exceptions. Tenant erasure, secret removal, and retention policy may require removal of original content. Preserve a minimal permitted audit fact where appropriate without retaining forbidden payloads in summaries, indexes, replicas, or backups. This is a technical policy boundary, not a claim about any particular legal requirement.

Experiment E06: restore a backup that predates an external effect, then reconcile before re-enabling dispatch. Experiment E07: delete an entity and expire eligible history while retaining a snapshot and delivering old messages. Verify reference safety, non-reuse of identities, and the documented handling of unavailable historical content.

Overload locality and reproducible verification

R10 Bound admitted work and preserve recovery progress

Set limits on runnable entities, per-entity inboxes, effect dispatch, WAL backlog, memory consumption, and cross-owner fan-out. Distinguish rejecting new work from durably accepting it for later execution. Once acceptance promises durability, pressure cannot silently discard the obligation. Document admission errors, retry guidance, and any cancellation or dead-letter policy.

Reserve progress for control traffic, lease renewal, recovery, and replication as appropriate to the protocol. Apply fairness across tenants and entities, not just worker threads. Watch queue age and tail latency alongside throughput. A single hot entity may saturate its authoritative owner even when other cores are idle; migration moves that bottleneck but does not parallelize its mutations. Partition application state only when the invariants permit it.

Admission feedback should combine SNC's own queue and WAL metrics with relevant host pressure. Linux PSI exposes CPU, memory, and I/O stall information [5]. Its usefulness depends on the execution path: polling or kernel-bypass work may require additional SNC instrumentation. Test memory pressure and slow storage before promising a particular limit.

R07 Extend the existing harness with controllable histories

Keep the existing crash-injection and physical-lab program. Add controllable time, network delivery, and storage completion at defined interfaces so a failing event history can be reproduced. Capture the random seed, event trace, build, topology, configuration, and failure model. A seed is useful only if the relevant nondeterminism is controlled.

Check operation histories against a small specification or reference model. Test safety, such as an invalid owner being unable to commit, separately from liveness, such as eventual progress after the modeled failures stop. Where both implementations exist, feed C++ and Rust equivalent histories and compare observable outcomes. A shared bug in both implementations still requires an independent oracle.

FoundationDB's peer-reviewed SIGMOD 2021 paper describes deterministic simulation as a central development practice [8]. It supports this verification approach, not a performance prediction for SNC. Simulation cannot establish hardware flush behavior, real scheduler latency, or the completeness of its own fault model; retain machine and network experiments for those boundaries.

Use small formal models where interleavings dominate: owner authority, placement commit versus HA promotion, wait registration, and effect claim versus cancellation. Introduce model checking only where the model is small enough to maintain and ties to implementation tests.

Experiment E08: overload one tenant while slowing WAL completion and replication. Verify bounded resources, declared admission behavior, and progress for protected workloads. Replay any failure before combining it with additional fault types.

Experiment plan and acceptance criteria

Run one boundary at a time before combining failures. Use a fault-free control under the same durability, topology, and resource settings. Record observations rather than assuming a faster run is more correct. Numeric latency and recovery thresholds should be agreed from the current baseline before the experiment.

Test	Setup and fault	Acceptance evidence
E01	Resume v1 work on mixed v1/v2 code; migrate ownership; test a rejected format.	Declared compatibility matrix holds; state and operation identity survive; incompatible execution fails explicitly.
E02	External action commits; receipt is lost; recover and retry with the same operation ID.	No duplicate effect where the destination contract guarantees deduplication; otherwise uncertainty is reconciled or surfaced.
E03	Delay a callback; cancel or supersede work; fail over; redeliver. Add clock changes.	Only eligible results commit; duplicates and late results follow policy; timer catch-up stays bounded.
E04	Crash between mutation and activation; race wait registration; rebuild an index.	Committed obligations remain recoverable; no lost eligible wakeup; index lag and snapshot guarantees match the API.
E05	Revoke authority while work waits; migrate; target another tenant; stall an adapter.	Authorization follows the declared revocation boundary; no unauthorized mutation; unrelated work continues.
E06	Restore an earlier backup in isolation; include effects already performed externally.	Coherent state and authority; no concurrent old-cluster authority; safe identity allocation and reconciled dispatch.
E07	Delete and collect history; retain snapshot roots; deliver stale messages and retries.	No unintended resurrection or premature reclamation; unavailable history is explicit; deduplication horizon remains valid.
E08	Sustain excess arrivals; slow storage and replicas; exercise one hot entity.	Bounded queues and memory; explicit admission response; no silent loss of accepted work; defined progress guarantees hold.

Measurements and evidence bundle

For each test capture offered, admitted, completed, and rejected work; p50, p95, p99, and worst observed latency; queue age; memory and WAL growth; recovery interruption; and correctness violations. Report zero observed violations only for the executed histories and fault model. A measured zero is not a universal proof.

Attach a manifest with commit IDs, executable hashes, versions, hardware and storage settings, topology, durability policy, dataset, seed or trace, exact commands, expected invariants, raw results, and interpretation. Capture successful controls and rejected compatibility cases as well as failures. Preserve a minimal reproducer for every new defect.

Agent work plan and review handoff

First return an evidence map

For each R01 through R14, classify the contract as implemented with evidence, implemented but insufficiently tested, partly implemented, missing, or superseded by an existing decision. Link the relevant source, tests, protocol description, and decision record. Treat missing documentation as a finding to investigate, not proof that the code lacks the behavior.

Use the existing project manifests and work-slice system. Inspect indirect dependencies and downstream behavior before proposing a local change. Preserve established ownership, locality, and module boundaries. Do not introduce a new database, messaging layer, scheduler, or framework merely because it appears in a reference.

Proposed sequence of work slices

Slice	Concrete output	Completion condition
A Map	R01–R14 evidence map and conflicts with current decisions.	Every item has evidence or a precise unresolved question.
B Specify	Acknowledgment and operation-state contracts; version and authority boundaries.	Implementer, reviewer, and QA can derive the same expected outcomes.
C Verify	Small E01 and E02 tests plus relevant existing core controls.	Results reproduce; disagreements are explained or tracked as defects.
D Extend	Activation, cancellation, restore, and overload experiments for implemented capabilities.	New tests use the same evidence and failure-model discipline.
E Reconcile	Targeted documentation changes and bounded implementation proposals.	Each proposal names the invariant, affected interfaces, test, and rollback or recovery behavior.

Follow the established three-way workflow: implementation, independent review, and QA. Within an authorized slice, address discovered defects on site and verify the result. If a finding changes architecture or exceeds the slice, record the dependency and proposed next action rather than silently broadening the work. This handoff is a review and planning input; use the project's existing authority for implementation decisions.

Questions that need a concrete answer

- Which durability and isolation guarantees does the current API actually promise, and where are they tested?
- What durable boundary represents suspended execution, and what compatibility metadata already exists?
- Which external destinations support idempotency, conditional operations, fencing, or authoritative readback?
- What is the supported retry horizon, retention policy, and process for restoring authority?
- Which entity and tenant limits are enforceable today, and which workloads expose the first bottleneck?

Requested response: evidence map, highest-impact unresolved decisions, proposed first work slice, its exact acceptance criteria, and any corrections to this addendum. Preserve a clear distinction between observed facts, inferred behavior, and proposed design.

Evidence and references

Reference [1] supplies the project baseline. References [2] through [7] are primary implementation documentation or engineering explanation; they are not peer-reviewed evidence of SNC behavior. Reference [8] is a peer-reviewed systems paper. No preprints are relied on in this addendum. The proposed contracts and experiments are engineering recommendations, not claims of measured SNC performance. Web references were checked on 3 November 2026.

1 SNC Architecture and Research Vision v0 2

Project document dated 3 November 2026. Relevant sections: implemented foundation 1; Rust transition 2; documents 4; scheduling and activation 5–6; snapshots and execution contracts 12–13; security 17; isolation 18; locality 19; roadmap 24; benchmarks 25; principles 27. Implementation statements remain project-reported until mapped to current code and tests.

Open the source architecture document

2 DBOS Upgrading Workflow Code

Official documentation. Describes patching and versioning for recovering long-running workflows across code changes. Supports the need for an explicit compatibility contract; does not establish an SNC implementation choice.

DBOS workflow upgrade documentation

3 DBOS Transactional Execution Beyond Durable Execution

Chuck Bear, 11 April 2024. Primary engineering explanation. Distinguishes atomically recorded database effects from external operations that require additional deduplication or reconciliation mechanisms.

DBOS transactional execution explanation

4 Microsoft Orleans Request scheduling

Official documentation. Explains single-threaded activations, request interleaving, and liveness tradeoffs. Used as a reference for scheduling semantics, not as a prescription for SNC's runtime.

Orleans request scheduling documentation

5 Linux Pressure Stall Information

Official kernel documentation by Johannes Weiner. Defines CPU, memory, and I/O pressure reporting. SNC still needs instrumentation for its own queues and execution paths.

Linux PSI documentation

6 PostgreSQL Reliability

Official documentation, version 18. Describes persistence and cache assumptions relevant to power-loss durability. Used as a storage reliability reference, not evidence that PostgreSQL owns SNC state.

PostgreSQL reliability documentation

7 Protocol Buffers Language Guide

Official documentation. Explains schema evolution and wire compatibility constraints. Used to motivate an explicit compatibility matrix independently of the chosen encoding.

Protocol Buffers message evolution guide

8 FoundationDB A Distributed Unbundled Transactional Key Value Store

Jingyu Zhou and coauthors. ACM SIGMOD 2021, industry track, pages 2653–2666. Peer-reviewed systems paper. Describes deterministic simulation and fault injection in a deployed database. Its operational experience is evidence for the method; transfer to SNC requires SNC-specific models and tests. DOI 10.1145/3448016.3457559.

FoundationDB paper